

PERANCANGAN DATA WAREHOUSE UNTUK ANALISIS PENJUALAN MENGGUNAKAN E-COMMERCE SALES DATASET

Dosen Pengampu:

Akhmad Irsyad, S.T., M.Kom.



Disusun Oleh:

Kelompok 11

GRACE VIES ANGEL	2409116005
ANGELA CAROLINE BUDIMAN	2409116008
ZAHRATURRAMADHANI	2409116014
MUHAMMAD FACHRI	2409116017

**PROGRAM STUDI SISTEM INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS MULAWARMAN**

2025

DAFTAR ISI

DAFTAR ISI	i
DAFTAR GAMBAR	iii
DAFTAR TABEL	iv
BAB I PENDAHULUAN	1
1.1. Latar Belakang	1
1.2. Rumusan Masalah	2
1.3. Tujuan Penelitian	2
1.4. Manfaat Penelitian	2
BAB II DATASET DAN PERANCANGAN DATA WAREHOUSE.....	4
2.1. Deskripsi Dataset	4
2.2. Struktur Data	4
2.3. Perancangan Data Warehouse.....	5
2.4. Desain Tabel.....	7
BAB III PROSES ETL	10
3.1. <i>Extract</i>	10
3.2. <i>Transform</i>	11
3.2.1. <i>Data Cleansing</i>	11
3.2.2. <i>Data Standardization</i>	18
3.2.3. <i>Data Validation</i>	24

3.2.4. Data <i>Enrichment</i>	31
3.2.5. Data <i>Filtering</i>	32
3.2.6. Data <i>Aggregation</i>	34
3.3. Data <i>Warehouse</i> (Star Schema).....	41
3.4. <i>Load</i>	44
BAB IV ANALISIS DATA	46
4.1. Analisis Jumlah Produk per Kategori	46
4.2. Analisis Rata-rata Harga per Kategori	47
4.3. Analisis Total <i>Review</i> per Kategori	48
4.4. Analisis Rata-rata <i>Rating</i> per Kategori	49
4.5. Analisis Top 10 Produk Terpopuler	51
4.6. Analisis Harga Setelah Diskon per Kategori.....	52
BAB V PENUTUP	54
5.1. Kesimpulan	54
5.2. Saran	54

DAFTAR GAMBAR

Gambar 2.1.	Skema Data Warehouse (<i>Star Schema</i>)	6
Gambar 4.1.	Analisis Jumlah Produk per Kategori.....	46
Gambar 4.2.	Analisis Rata-rata Harga per Kategori.....	47
Gambar 4.3.	Analisis Total <i>Review</i> per Kategori	48
Gambar 4.4.	Analisis Rata-rata <i>Rating</i> per Kategori.....	50
Gambar 4.5.	Analisis Top 10 Produk Terpopuler.....	51
Gambar 4.6.	Analisis Harga Setelah Diskon per Kategori	52

DAFTAR TABEL

Tabel 2.1.	Struktur Dataset.....	4
Tabel 2.2.	<i>Fact Table</i>	8
Tabel 2.3.	<i>Dimension Category</i>	8
Tabel 2.4.	<i>Dimension Product</i>	8

BAB I

PENDAHULUAN

1.1. Latar Belakang

Perkembangan teknologi informasi mendorong pesatnya pertumbuhan e-commerce yang menghasilkan data transaksi dalam jumlah besar. Data tersebut mencakup berbagai informasi seperti kategori produk, jumlah pembelian, harga, metode pengiriman, dan status pesanan. Data ini memiliki potensi besar untuk dianalisis dalam mendukung pengambilan keputusan bisnis yang lebih efektif.

Dataset yang digunakan dalam penelitian ini berasal dari kaggle dengan judul *E-Commerce Sales Data*. Dataset ini terdiri dari beberapa file seperti Amazon Sale Report, Sale Report, P & L dataset, International Sales Report, Cloud Warehouse Compersion Chart, Expense IIGF, dan May-2022 yang mencerminkan berbagai aspek penjualan. Namun, data masih dalam kondisi mentah, tersebar di beberapa file, dan belum memiliki struktur yang seragam sehingga sulit untuk langsung dianalisis.

Untuk mengatasi hal tersebut, kami menerapkan Business Intelligence, yang di mana data warehouse dimanfaatkan sebagai tempat penyimpanan data yang terintegrasi dan terstruktur. Sebelum data dimasukkan ke dalam data warehouse, dilakukan proses ETL yang meliputi extract, transform, dan load agar data menjadi lebih bersih dan konsisten.

Dengan adanya data warehouse, data yang sebelumnya tidak terstruktur dapat diolah menjadi informasi yang lebih mudah dianalisis. Hal ini akan memperoleh insight seperti pola penjualan, kategori produk yang paling diminati, serta performa penjualan, sehingga dapat membantu dalam pengambilan keputusan bisnis e-commerce.

1.2. Rumusan Masalah

Berdasarkan uraian sebelumnya, dalam penelitian ini, terdapat beberapa permasalahan yang menjadi fokus pembahasan, yaitu:

1. Bagaimana mengolah data penjualan e-commerce yang masih tersebar dan belum terstruktur agar dapat digunakan untuk analisis?
2. Bagaimana proses ETL dilakukan untuk memperbaiki kualitas data sehingga menjadi lebih bersih dan konsisten?
3. Bagaimana menyusun data warehouse yang mampu mengintegrasikan data penjualan ke dalam satu struktur yang lebih terorganisir?
4. Bagaimana hasil pengolahan data dapat memberikan gambaran mengenai kondisi dan pola penjualan?

1.3. Tujuan Penelitian

Berdasarkan permasalahan yang telah dirumuskan, penelitian ini memiliki beberapa tujuan sebagai berikut:

1. Mengolah data penjualan e-commerce yang masih tersebar agar menjadi lebih terstruktur dan siap digunakan untuk analisis.
2. Menerapkan proses ETL untuk meningkatkan kualitas data sehingga menjadi lebih bersih dan konsisten.
3. Merancang data warehouse yang dapat mengintegrasikan data penjualan ke dalam satu sistem yang terorganisir.
4. Menghasilkan informasi yang dapat memberikan gambaran mengenai kondisi dan pola penjualan.

1.4. Manfaat Penelitian

Penelitian ini diharapkan dapat memberikan beberapa manfaat, baik dalam memahami pengolahan data maupun penerapannya dalam analisis penjualan, sebagai berikut:

1. Bagi penulis, penelitian ini dapat menambah pemahaman dalam pengolahan data serta penerapan data warehouse dalam Business Intelligence.
2. Bagi pembaca, penelitian ini dapat memberikan gambaran mengenai bagaimana data penjualan e-commerce diolah menjadi informasi yang lebih terstruktur.
3. Bagi penelitian selanjutnya, penelitian ini dapat menjadi referensi dalam pengembangan analisis data dan perancangan data warehouse pada bidang serupa.

BAB II

DATASET DAN PERANCANGAN DATA WAREHOUSE

2.1. Deskripsi Dataset

Dataset yang digunakan dalam penelitian ini diperoleh dari kaggle dengan judul *E-Commerce Sales Data*. Dataset ini terdiri dari beberapa file yang berisi data terkait aktivitas penjualan pada platform e-commerce, mulai dari transaksi, informasi produk, hingga penjualan dari berbagai wilayah.

Salah satu file yang digunakan adalah amazon sale report, yang memuat data transaksi seperti kategori produk, waktu pemesanan, status pesanan, jumlah barang yang dibeli, serta total nilai penjualan. Data tersebut menjadi fokus utama karena berkaitan langsung dengan aktivitas penjualan yang terjadi.

Secara keseluruhan, data yang tersedia belum dalam kondisi siap pakai karena masih terpisah dalam beberapa file dan memiliki format yang belum seragam. Selain itu, terdapat beberapa bagian data yang perlu disesuaikan agar dapat digunakan dengan lebih baik. Oleh karena itu, diperlukan proses pengolahan agar data menjadi lebih rapi dan dapat dimanfaatkan dalam analisis.

2.2. Struktur Data

Dataset yang digunakan dalam penelitian ini memuat berbagai atribut yang berkaitan dengan transaksi penjualan pada e-commerce. Setiap atribut memberikan informasi yang berbeda, seperti data pesanan, produk, hingga pengiriman. Struktur data ini digunakan sebagai dasar dalam memahami isi dataset sebelum dilakukan proses analisis lebih lanjut.

Tabel 2.1. Struktur Dataset

No	Atribut	Format Data	Deskripsi
1	order_id	Teks	Kode unik setiap transaksi
2	date	Tanggal	Waktu terjadinya transaksi
3	status	Teks	Status pesanan saat ini
4	fulfilment	Teks	Metode pemenuhan pesanan
5	sales_channel	Teks	Jalur penjualan produk
6	category	Teks	Jenis atau kategori produk
7	size	Teks	Ukuran produk
8	sku	Teks	Stock Keeping Unit (Kode unik produk)
9	asin	Teks	Amazon Standard Identification Number
10	courier_status	Teks	Status pengiriman oleh kurir
11	qty	Angka	Jumlah produk yang dibeli
12	amount	Desimal	Total nilai transaksi
13	ship_city	Teks	Kota tujuan pengiriman
14	ship_state	Teks	Provinsi tujuan
15	ship_postal_code	Teks	Kode pos
16	ship_country	Teks	Negara tujuan
17	promotion	Teks	Informasi atau kode promosi
18	b2b	Teks	Jenis transaksi (B2B/B2C)

Berdasarkan struktur tersebut, dataset terdiri dari berbagai tipe data, seperti teks, angka, dan tanggal. Namun, masih diperlukan proses pengolahan data agar setiap atribut memiliki format yang sesuai dan dapat digunakan dalam analisis.

2.3. Perancangan Data Warehouse

Perancangan data warehouse pada penelitian ini dilakukan untuk menyusun kembali data yang telah dibersihkan agar lebih rapi dan mudah digunakan dalam proses analisis. Data yang sebelumnya masih tersebar kemudian diorganisir ke dalam bentuk tabel yang saling terhubung.

Dalam penelitian ini digunakan model *star schema*, yaitu struktur yang memiliki satu tabel utama sebagai pusat data dan beberapa tabel pendukung. Struktur ini dipilih karena lebih sederhana dan mudah dipahami, sehingga memudahkan dalam melihat hubungan antara data penjualan dengan informasi lain seperti produk, waktu, dan lokasi.

1. Fact Table (fact_sales)

Tabel fakta merupakan tabel utama yang berisi data transaksi penjualan. Tabel ini menjadi pusat dari data warehouse karena menyimpan data yang akan dianalisis, seperti jumlah penjualan dan nilai transaksi.

Pada penelitian ini, tabel fact_sales digunakan untuk menyimpan data utama penjualan. Selain itu, tabel ini juga memiliki beberapa atribut penghubung yang

digunakan untuk mengakses informasi dari tabel dimensi.

Atribut yang terdapat pada tabel ini antara lain:

- *product_id* untuk menghubungkan ke data produk
- *time_id* untuk menghubungkan ke data waktu
- *location_id* untuk menghubungkan ke data lokasi
- *qty* yang menunjukkan jumlah produk yang terjual
- *amount* yang menunjukkan total nilai penjualan

2. Dimension Table

Tabel dimensi digunakan untuk menyimpan informasi pendukung yang membantu menjelaskan data pada tabel fakta. Dengan adanya tabel dimensi, data dapat dianalisis dari berbagai sisi.

- *dim_product*

Tabel ini berisi informasi terkait produk yang dijual. Data pada tabel ini membantu dalam mengetahui jenis produk serta kategori yang tersedia, sehingga analisis dapat dilakukan berdasarkan produk atau kategori tertentu.

- *dim_time*

Tabel ini menyimpan informasi waktu terjadinya transaksi. Data ini digunakan untuk melihat pola penjualan berdasarkan periode waktu, seperti per bulan atau per tahun.

- *dim_location*

Tabel ini berisi informasi terkait lokasi pengiriman. Data lokasi membantu dalam menganalisis penyebaran penjualan berdasarkan wilayah.

3. Relasi Antar Tabel

Relasi antar tabel digunakan untuk menghubungkan tabel fakta dengan tabel dimensi. Pada perancangan ini, tabel *fact_sales* terhubung dengan *dim_product* melalui *product_id*, dengan *dim_time* melalui *time_id*, dan dengan *dim_location* melalui *location_id*.

Dengan adanya hubungan tersebut, setiap data transaksi dapat dikaitkan dengan informasi produk, waktu, dan lokasi secara bersamaan. Hal ini membuat proses analisis menjadi lebih mudah dan fleksibel.

2.4. Desain Tabel

Pada tahap ini dilakukan penyusunan struktur tabel yang akan digunakan dalam data warehouse. Setiap tabel dirancang sesuai dengan fungsinya, yaitu sebagai tabel fakta yang

menyimpan data utama dan tabel dimensi yang menyimpan informasi pendukung.

Tabel 2.2 Fact Table (fact_sales)

No	Atribut	Jenis Data	Deskripsi
1	sales_id	Integer	ID unik untuk setiap data penjualan
2	product_id	Integer	Kode produk yang terhubung ke dim_product
3	time_id	Integer	Kode waktu yang terhubung ke dim_time
4	location_id	Integer	Kode lokasi yang terhubung ke dim_location
5	qty	Integer	Jumlah produk yang terjual
6	amount	Float	Total nilai transaksi penjualan

Tabel ini digunakan sebagai pusat data karena berisi informasi utama terkait transaksi penjualan. Data pada tabel ini nantinya digunakan dalam proses analisis karena bersifat numerik dan dapat diolah.

Tabel 2.3 Dimension Product (dim_product)

No	Atribut	Jenis Data	Deskripsi
1	product_id	Integer	ID unik produk
2	product	Object (Teks)	Nama produk
3	category	Object (Teks)	Kategori produk

Tabel ini berfungsi untuk menyimpan informasi terkait produk yang dijual. Data pada tabel ini membantu dalam mengelompokkan produk berdasarkan kategori tertentu.

Tabel 2.4 Dimension Time (dim_time)

No	Atribut	Jenis Data	Deskripsi
1	time_id	Integer	ID waktu
2	date	Object (Tanggal)	Tanggal transaksi
3	month	Integer	Bulan transaksi
4	year	Integer	Tahun transaksi

Tabel ini digunakan untuk menyimpan informasi waktu transaksi sehingga analisis dapat dilakukan berdasarkan periode tertentu.

Tabel 2.5 Dimension Location (dim_location)

No	Atribut	Jenis Data	Deskripsi
1	location_id	Integer	ID lokasi
2	city	Object (Teks)	Kota
3	state	Object (Teks)	Provinsi
4	country	Object (Teks)	Negara

Tabel ini menyimpan informasi terkait lokasi pengiriman produk, yang dapat digunakan untuk melihat distribusi penjualan berdasarkan wilayah.

BAB III

PROSES ETL

3.1. *Extract*

Tahap *Extract* merupakan tahap awal dalam proses ETL yang bertujuan untuk mengambil data dari sumber data yang digunakan dalam penelitian. Pada penelitian ini data diperoleh dari file Amazon Sale Report.csv yang kemudian diimpor ke dalam Google Colab menggunakan *library Pandas*.

```
import pandas as pd
```

```
#Load data
```

```
df_raw = pd.read_csv('/content/drive/MyDrive/praktikum/Amazon Sale Report.csv')
```

```
df_raw.head()
```

Index	Order ID	Date	Status	Fulfilment	Sales Channel	ship-service-level	Style	SKU	Category	...	currency	Amount	ship-city	ship-state	ship-postal-code	ship-country	promotion-ids	B2B	fulfilled-by	Unnamed: 22	
0	0	405-8078784-5731545	04-30-22	Cancelled	Merchant	Amazon.in	Standard	SET388-KR-NP-S	Set	...	INR	647.82	MUMBAI	MAHARASHTRA	400081.0	IN	NaN	False	Easy Ship	NaN	
1	1	171-9108151-1101148	04-30-22	Shipped - Delivered to Buyer	Merchant	Amazon.in	Standard	JNE3781-KR-XXXL	kurta	...	INR	406.00	BENGALURU	KARNATAKA	560085.0	IN	Amazon PLCC Free-Financing Universal Merchant ...	False	Easy Ship	NaN	
2	2	404-0687878-7273148	04-30-22	Shipped	Amazon	Amazon.in	Expedited	JNE3371-KR-XL	kurta	...	INR	329.00	NAVI MUMBAI	MAHARASHTRA	410210.0	IN	IN Core Free Shipping 2015/04/08 23-48-5-108	True	NaN	NaN	
3	3	483-9615377-8133951	04-30-22	Cancelled	Merchant	Amazon.in	Standard	J0341	J0341-DRL	Western Dress	...	INR	753.33	PUDUCHERRY	PUDUCHERRY	605008.0	IN	NaN	False	Easy Ship	NaN
4	4	407-1069700-7240320	04-30-22	Shipped	Amazon	Amazon.in	Expedited	JNE3671	JNE3671-TU-XXXL	Top	...	INR	574.00	CHENNAI	TAMIL NADU	600073.0	IN	NaN	False	NaN	NaN

5 rows × 24 columns

5 rows x 24 columns

Interpretasi:

Kode tersebut digunakan untuk membaca file Amazon Sale Report.csv menggunakan library Pandas dan menyimpannya ke dalam bentuk dataframe dengan nama variabel df_raw. Setelah dataset berhasil dimuat, perintah df_raw.head() digunakan untuk menampilkan beberapa baris pertama dari data.

Dari tampilan tersebut dapat dilihat contoh isi pada setiap kolom seperti Order ID, Date, Status, Fulfilment, Sales Channel, Category, Size, Amount, ship-city, ship-state, dan ship-country. Tampilan awal ini membantu untuk memahami struktur serta isi dataset penjualan Amazon sebelum dilakukan proses pembersihan dan pengolahan data pada tahap selanjutnya.

3.2. Transform

Tahap *transform* merupakan proses pengolahan data setelah data berhasil diambil dari sumber dataset. Pada tahap ini dilakukan beberapa proses seperti pengecekan kondisi data, pembersihan data, serta penyesuaian format data agar lebih terstruktur dan siap digunakan pada tahap analisis.

3.2.1. Data Cleansing

Data *cleansing* merupakan proses pembersihan data untuk memperbaiki atau menghilangkan data yang tidak lengkap, tidak konsisten, atau tidak relevan. Pada tahap ini dilakukan beberapa langkah seperti menangani nilai kosong (*missing value*), menghapus data duplikat, serta memastikan format data lebih konsisten. Proses ini bertujuan agar dataset menjadi lebih bersih dan siap digunakan untuk tahap analisis selanjutnya.

#Cek data kosong

```
df_raw.isnull().sum()
```

```
...
index      0
Order ID   0
Date       0
Status     0
Fulfilment 0
Sales Channel 0
ship-service-level 0
Style      0
SKU        0
Category   0
Size       0
ASIN       0
Courier Status  6872
Qty         0
currency    7795
Amount      7795
ship-city    33
ship-state   33
ship-postal-code 33
ship-country 33
promotion-ids 49153
B2B         0
fulfilled-by 89698
Unnamed: 22  49050
dtype: int64
```

Interpretasi:

Kode tersebut digunakan untuk mengecek jumlah data kosong (missing values) pada setiap

kolom di dalam dataframe `df_raw` dengan menggunakan perintah `df_raw.isnull().sum()`. Hasil output menunjukkan jumlah nilai kosong pada masing-masing atribut dataset, seperti pada kolom Courier Status, currency, Amount, promotion-ids, dan fulfilled-by yang masih memiliki data kosong, sedangkan beberapa kolom lainnya tidak memiliki missing values. Proses ini dilakukan untuk mengetahui kondisi data sebelum masuk ke tahap pembersihan dan pengolahan data lebih lanjut.

#Hapus data kosong

```
df_clean = df_raw.dropna().copy()
```

Cek ulang data

```
df_clean.isnull().sum()
```

```
...
0
index      0
Order ID   0
Date       0
Status     0
Fulfilment 0
Sales Channel 0
ship-service-level 0
Style      0
SKU        0
Category   0
Size       0
ASIN       0
Courier Status 0
Qty        0
currency   0
Amount     0
ship-city  0
ship-state 0
ship-postal-code 0
ship-country 0
promotion-ids 0
B2B        0
fulfilled-by 0
Unnamed: 22 0
dtype: int64
```

Inrepretasi:

Kode `df_raw.dropna().copy()` digunakan untuk menghapus seluruh baris data yang mengandung nilai kosong dari dataframe `df_raw`, kemudian hasilnya disimpan ke dalam dataframe baru bernama `df_clean`. Proses ini bertujuan untuk membersihkan dataset agar

data yang digunakan pada tahap analisis selanjutnya lebih lengkap, konsisten, dan mengurangi kemungkinan kesalahan dalam pengolahan data.

Setelah proses penghapusan data kosong dilakukan, digunakan kembali perintah `df_clean.isnull().sum()` untuk mengecek ulang apakah masih terdapat missing values pada dataset yang telah dibersihkan. Hasil output menunjukkan seluruh kolom memiliki nilai 0, yang berarti tidak ada lagi data kosong pada dataframe `df_clean` sehingga dataset sudah siap digunakan pada tahap analisis berikutnya.

```
# CEK JUMLAH DATA SEBELUM HAPUS DUPLIKAT
```

```
print("=== BEFORE DROP DUPLICATES ===")
```

```
print("Jumlah data:", len(df_clean))
```

```
print("Jumlah data duplikat:", df_clean.duplicated().sum())
```

```
***  === BEFORE DROP DUPLICATES ===  
      Jumlah data: 19379  
      Jumlah data duplikat: 0
```

Interpretasi:

Kode `len(df_clean)` dan `df_clean.duplicated().sum()` digunakan untuk mengecek jumlah seluruh data serta jumlah data duplikat pada dataframe `df_clean` sebelum dilakukan proses penghapusan data duplikat. Output menunjukkan bahwa dataset memiliki 19.379 baris data dan tidak terdapat data duplikat karena jumlah duplikat bernilai 0. Proses ini dilakukan untuk memvalidasi kondisi dataset sebelum tahap data cleansing dilakukan.

```
# HAPUS DATA DUPLIKAT
```

```
df_clean = df_clean.drop_duplicates()
```

```
# CEK JUMLAH DATA SETELAH HAPUS DUPLIKAT
```

```
print("=== AFTER DROP DUPLICATES ===")
```

```
print("Jumlah data:", len(df_clean))
```

```
print("Jumlah data duplikat:", df_clean.duplicated().sum())
```

```
*** === AFTER DROP DUPLICATES ===  
Jumlah data: 19379  
Jumlah data duplikat: 0
```

Interpretasi:

Kode `df_clean = df_clean.drop_duplicates()` digunakan untuk menghapus data duplikat pada dataframe `df_clean`. Proses ini bertujuan untuk membersihkan dataset dari data yang berulang agar hasil analisis menjadi lebih akurat dan tidak terjadi redundansi data.

Selanjutnya, perintah `len(df_clean)` dan `df_clean.duplicated().sum()` digunakan untuk mengecek jumlah data serta memastikan tidak terdapat data duplikat setelah proses penghapusan dilakukan. Output menunjukkan bahwa dataset memiliki 19.379 data dan jumlah data duplikat bernilai 0, sehingga dataset sudah bersih dari data yang berulang.

3.2.2. Data Standardization

Data *standardization* merupakan proses menyeragamkan format data agar memiliki bentuk yang konsisten dan lebih mudah digunakan dalam proses analisis. Pada tahap ini dilakukan penyesuaian terhadap beberapa kolom yang masih memiliki format teks atau simbol tertentu.

Proses ini dilakukan dengan membersihkan simbol atau teks yang tidak diperlukan dan mengubahnya menjadi format data yang lebih sesuai, misalnya mengubah nilai harga menjadi numerik. Selain itu, standarisasi juga membantu mengurangi perbedaan format data yang dapat menyebabkan kesalahan saat proses analisis. Dengan adanya proses standarisasi, data menjadi lebih terstruktur dan konsisten.

```
# RENAME KOLOM
```

```
df_clean.columns = df_clean.columns.str.lower().str.replace(' ', '_')
```

```
# CEK KOLOM SETELAH RENAME
```

```
print("=== AFTER RENAME ===")

print(df_clean.columns)
```

```
*** === AFTER RENAME ===
Index(['index', 'order_id', 'date', 'status', 'fulfilment', 'sales_channel_',
      'ship-service-level', 'style', 'sku', 'category', 'size', 'asin',
      'courier_status', 'qty', 'currency', 'amount', 'ship-city',
      'ship-state', 'ship-postal-code', 'ship-country', 'promotion-ids',
      'b2b', 'fulfilled-by', 'unnamed:_22'],
      dtype='object')
```

Interpretasi:

Kode `df_clean.columns = df_clean.columns.str.lower().str.replace(' ', '_')` digunakan untuk menyeragamkan format nama kolom dengan mengubah seluruh huruf menjadi huruf kecil dan mengganti spasi menjadi underscore (_). Proses ini bertujuan agar penamaan kolom lebih konsisten dan memudahkan proses pengolahan data.

Selanjutnya, perintah `print(df_clean.columns)` digunakan untuk menampilkan hasil nama kolom setelah proses rename dilakukan. Output menunjukkan bahwa nama kolom berhasil diubah menjadi format yang lebih seragam, seperti Order ID menjadi `order_id`, Sales Channel menjadi `sales_channel_`, dan Courier Status menjadi `courier_status`.

UBAH FORMAT TANGGAL

```
df_clean['date'] = pd.to_datetime(df_clean['date'])
```

CEK FORMAT TANGGAL SETELAH DIUBAH

```
print("=== AFTER DATE FORMAT ===")
```

```
print(df_clean['date'].head())
```

```
print(df_clean['date'].dtype)
```

```

*** === AFTER DATE FORMAT ===
49051    2022-05-31
49077    2022-05-31
49081    2022-05-31
49082    2022-05-31
49083    2022-05-31
Name: date, dtype: datetime64[ns]
datetime64[ns]

```

Interpretasi:

Kode `df_clean['date'] = pd.to_datetime(df_clean['date'])` digunakan untuk mengubah tipe data pada kolom date menjadi format datetime agar data tanggal dapat diproses dan dianalisis dengan lebih mudah.

Selanjutnya, perintah `print(df_clean['date'].head())` dan `print(df_clean['date'].dtype)` digunakan untuk menampilkan beberapa data awal serta mengecek tipe data pada kolom date setelah proses konversi dilakukan. Output menunjukkan bahwa format tanggal berhasil diubah menjadi `datetime64[ns]`.

```
# PASTIKAN HARGA ANGKA
```

```
df_clean['amount'] = pd.to_numeric(df_clean['amount'], errors='coerce')
```

```
# CEK TIPE DATA AMOUNT SETELAH DIUBAH
```

```
print("=== AFTER CONVERT AMOUNT ===")
```

```
print(df_clean['amount'].head())
```

```
print(df_clean['amount'].dtype)
```

```

=== AFTER CONVERT AMOUNT ===
49051    888.0
49077    371.0
49081    771.0
49082     0.0
49083    690.0
Name: amount, dtype: float64
float64

```

Interpretasi:

Kode `df_clean['amount'] = pd.to_numeric(df_clean['amount'], errors='coerce')` digunakan untuk mengubah data pada kolom `amount` menjadi tipe numerik agar dapat digunakan dalam proses perhitungan dan analisis statistik.

Selanjutnya, perintah `print(df_clean['amount'].head())` dan `print(df_clean['amount'].dtype)` digunakan untuk menampilkan beberapa data awal serta mengecek tipe data pada kolom `amount` setelah proses konversi dilakukan. Output menunjukkan bahwa tipe data kolom `amount` berhasil menjadi `float64`.

Perbaikan Struktur

```
df_clean.columns = df_clean.columns.str.replace('-', '_')
```

Pengecekan Struktur Kolom Setelah diperbaiki

```
df_clean.columns
```

```
*** Index(['index', 'order_id', 'date', 'status', 'fulfilment', 'sales_channel_',
          'ship_service_level', 'style', 'sku', 'category', 'size', 'asin',
          'courier_status', 'qty', 'currency', 'amount', 'ship_city',
          'ship_state', 'ship_postal_code', 'ship_country', 'promotion_ids',
          'b2b', 'fulfilled_by', 'unnamed:_22', 'year', 'month'],
          dtype='object')
```

Interpretasi:

Kode `df_clean.columns = df_clean.columns.str.replace('-', '_')` digunakan untuk memperbaiki dan menyeragamkan struktur nama kolom dengan mengganti tanda hubung (-) menjadi underscore (_). Proses ini bertujuan agar format penamaan kolom lebih konsisten dan memudahkan proses pengolahan data.

Selanjutnya, perintah `df_clean.columns` digunakan untuk menampilkan hasil nama kolom setelah proses perbaikan dilakukan. Output menunjukkan bahwa nama kolom yang sebelumnya menggunakan tanda hubung, seperti `ship-service-level`, `ship-city`, dan `promotion-ids`, berhasil diubah menjadi `ship_service_level`, `ship_city`, dan `promotion_ids`.

3.2.3. Data Validation

Data *validation* merupakan proses untuk memastikan bahwa data yang telah dibersihkan dan distandarisasi memiliki nilai yang valid serta sesuai dengan kebutuhan analisis. Pada tahap ini dilakukan pengecekan terhadap tipe data, nilai yang tidak wajar, serta memastikan setiap kolom memiliki format data yang sesuai. Proses validasi penting dilakukan untuk memastikan bahwa data yang digunakan sudah benar dan tidak menimbulkan kesalahan pada saat proses analisis.

Cek tipe data

df_clean.info()

```
*** <class 'pandas.core.frame.DataFrame'>
Index: 19379 entries, 49051 to 128891
Data columns (total 24 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   index                 19379 non-null  int64
1   Order ID              19379 non-null  object
2   Date                  19379 non-null  object
3   Status                19379 non-null  object
4   Fulfilment            19379 non-null  object
5   Sales Channel         19379 non-null  object
6   ship-service-level    19379 non-null  object
7   Style                 19379 non-null  object
8   SKU                   19379 non-null  object
9   Category              19379 non-null  object
10  Size                  19379 non-null  object
11  ASIN                  19379 non-null  object
12  Courier Status         19379 non-null  object
13  Qty                   19379 non-null  int64
14  currency              19379 non-null  object
15  Amount                19379 non-null  float64
16  ship-city              19379 non-null  object
17  ship-state            19379 non-null  object
18  ship-postal-code      19379 non-null  float64
19  ship-country          19379 non-null  object
20  promotion-ids         19379 non-null  object
21  B2B                   19379 non-null  bool
22  fulfilled-by          19379 non-null  object
23  Unnamed: 22           19379 non-null  object
dtypes: bool(1), float64(2), int64(2), object(19)
memory usage: 3.6+ MB
```

Interpretasi:

Kode df_clean.info() digunakan untuk menampilkan informasi lengkap mengenai dataset setelah proses pembersihan data dilakukan. Output yang dihasilkan menampilkan jumlah baris dan kolom, nama atribut, jumlah data non-null, tipe data setiap kolom, serta penggunaan memori dataset.

Berdasarkan hasil output, dataset memiliki 19.379 baris dan 24 kolom dengan tipe data seperti object, int64, float64, dan bool. Selain itu, seluruh kolom menunjukkan jumlah non-

null yang sama, sehingga menandakan tidak terdapat lagi data kosong pada dataset. Proses ini dilakukan untuk memvalidasi bahwa struktur dan tipe data sudah sesuai sebelum masuk ke tahap analisis data lebih lanjut.

```
# Lihat statistik
```

```
df_clean.describe()
```

```
***
```

	index	Qty	Amount	ship-postal-code
count	19379.000000	19379.000000	19379.000000	19379.000000
mean	86650.405542	1.004799	673.245162	464618.933743
std	22204.535858	0.082081	287.866514	191536.937504
min	49051.000000	1.000000	0.000000	110001.000000
25%	67813.000000	1.000000	471.000000	380058.000000
50%	83317.000000	1.000000	678.000000	500032.000000
75%	105153.500000	1.000000	786.000000	600039.000000
max	128891.000000	5.000000	2796.000000	855107.000000

Interpretasi:

Kode `df_clean.describe()` digunakan untuk menampilkan statistik deskriptif dari kolom numerik pada dataset, seperti Qty, Amount, dan ship-postal-code. Output yang dihasilkan menampilkan informasi statistik seperti jumlah data (count), rata-rata (mean), standar deviasi (std), nilai minimum (min), kuartil data (25%, 50%, 75%), serta nilai maksimum (max). Proses ini dilakukan untuk memvalidasi kondisi data numerik dan membantu memahami karakteristik dataset sebelum dilakukan analisis lebih lanjut.

```
# CEK KOLOM SEBELUM RENAME
```

```
print("=== BEFORE RENAME ===")
```

```
print(df_clean.columns)
```

```

*** === BEFORE RENAME ===
Index(['index', 'Order ID', 'Date', 'Status', 'Fulfilment', 'Sales Channel ',
      'ship-service-level', 'Style', 'SKU', 'Category', 'Size', 'ASIN',
      'Courier Status', 'Qty', 'currency', 'Amount', 'ship-city',
      'ship-state', 'ship-postal-code', 'ship-country', 'promotion-ids',
      'B2B', 'fulfilled-by', 'Unnamed: 22'],
      dtype='object')

```

Interpretasi:

Kode `print("=== BEFORE RENAME ===")` dan `print(df_clean.columns)` digunakan untuk menampilkan daftar nama kolom pada dataframe `df_clean` sebelum dilakukan proses rename atau standardisasi nama kolom. Output yang dihasilkan memperlihatkan seluruh atribut yang terdapat pada dataset, seperti Order ID, Date, Status, Fulfilment, Sales Channel, Amount, ship-city, dan atribut lainnya. Proses ini bertujuan untuk memvalidasi struktur dan nama kolom pada dataset sebelum dilakukan perubahan format penamaan kolom pada tahap standardisasi data.

CEK FORMAT TANGGAL SEBELUM DIUBAH

```

print("=== BEFORE DATE FORMAT ===")

print(df_clean['date'].head())

print(df_clean['date'].dtype)

```

```

*** === BEFORE DATE FORMAT ===
49051    05-31-22
49077    05-31-22
49081    05-31-22
49082    05-31-22
49083    05-31-22
Name: date, dtype: object
object

```

Interpretasi:

Kode `print(df_clean['date'].head())` dan `print(df_clean['date'].dtype)` digunakan untuk menampilkan beberapa data awal pada kolom date serta mengecek tipe data dari kolom tersebut sebelum dilakukan perubahan format tanggal. Output menunjukkan bahwa data tanggal masih berbentuk teks dengan tipe data object. Proses ini dilakukan untuk

memvalidasi kondisi awal kolom date sehingga dapat diketahui bahwa kolom tersebut masih perlu diubah ke format datetime pada tahap standardisasi data.

```
# CEK TIPE DATA AMOUNT SEBELUM DIUBAH
```

```
print("=== BEFORE CONVERT AMOUNT ===")
```

```
print(df_clean['amount'].head())
```

```
print(df_clean['amount'].dtype)
```

```
***  === BEFORE CONVERT AMOUNT ===  
49051    888.0  
49077    371.0  
49081    771.0  
49082      0.0  
49083    690.0  
Name: amount, dtype: float64  
float64
```

Interpretasi:

Kode `print(df_clean['amount'].head())` dan `print(df_clean['amount'].dtype)` digunakan untuk menampilkan beberapa data awal pada kolom amount serta mengecek tipe data dari kolom tersebut sebelum dilakukan proses konversi data. Output menunjukkan contoh nilai pada kolom amount dan tipe data yang digunakan yaitu float64. Proses ini dilakukan untuk memvalidasi kondisi data pada kolom amount sebelum dilakukan pengolahan atau standardisasi data lebih lanjut.

```
# CEK JUMLAH DATA SEBELUM HAPUS DUPLIKAT
```

```
print("=== BEFORE DROP DUPLICATES ===")
```

```
print("Jumlah data:", len(df_clean))
```

```
print("Jumlah data duplikat:", df_clean.duplicated().sum())
```

```
*** === BEFORE DROP DUPLICATES ===  
Jumlah data: 19379  
Jumlah data duplikat: 0
```

Interpretasi:

Kode `len(df_clean)` dan `df_clean.duplicated().sum()` digunakan untuk mengecek jumlah seluruh data serta jumlah data duplikat pada dataframe `df_clean` sebelum dilakukan proses penghapusan data duplikat. Output menunjukkan bahwa dataset memiliki 19.379 baris data dan tidak terdapat data duplikat karena jumlah duplikat bernilai 0. Proses ini dilakukan untuk memvalidasi kondisi dataset sebelum tahap data cleansing dilakukan.

```
# CEK KOLOM SEBELUM DITAMBAHKAN
```

```
print("=== BEFORE ADD NEW COLUMN ===")
```

```
print(df_clean.columns)
```

```
*** === BEFORE ADD NEW COLUMN ===  
Index(['index', 'order_id', 'date', 'status', 'fulfilment', 'sales_channel_',  
       'ship-service-level', 'style', 'sku', 'category', 'size', 'asin',  
       'courier_status', 'qty', 'currency', 'amount', 'ship-city',  
       'ship-state', 'ship-postal-code', 'ship-country', 'promotion-ids',  
       'b2b', 'fulfilled-by', 'unnamed:_22'],  
      dtype='object')
```

Interpretasi:

Kode `print(df_clean.columns)` digunakan untuk menampilkan daftar nama kolom pada dataframe `df_clean` sebelum dilakukan penambahan kolom baru. Output yang dihasilkan memperlihatkan seluruh atribut yang terdapat pada dataset, seperti `order_id`, `date`, `status`, `amount`, `ship-city`, dan atribut lainnya. Proses ini dilakukan untuk memvalidasi struktur dataset serta memastikan kolom yang tersedia sebelum tahap penambahan kolom baru dilakukan pada proses data enrichment.

3.2.4. Data Enrichment

Data *enrichment* merupakan proses penambahan atau pengembangan informasi pada dataset agar data menjadi lebih lengkap dan mudah digunakan dalam analisis. Pada tahap ini dilakukan pembuatan beberapa atribut baru berdasarkan pengolahan data yang sudah ada.

Proses ini dilakukan dengan mengolah nilai-nilai yang tersedia menjadi informasi baru yang lebih ringkas dan mudah dipahami, seperti hasil perhitungan serta pengelompokan data ke dalam beberapa kategori. Dengan cara ini, data yang awalnya masih berupa nilai mentah dapat diubah menjadi informasi yang lebih terstruktur.

```
# TAMBAH KOLOM BARU

df_clean['year'] = df_clean['date'].dt.year

df_clean['month'] = df_clean['date'].dt.month


# CEK KOLOM SETELAH DITAMBAHKAN

print("=== AFTER ADD NEW COLUMN ===")

print(df_clean[['date', 'year', 'month']].head())

***  === AFTER ADD NEW COLUMN ===
      date  year  month
49051 2022-05-31  2022     5
49077 2022-05-31  2022     5
49081 2022-05-31  2022     5
49082 2022-05-31  2022     5
49083 2022-05-31  2022     5
```

Interpretasi:

Kode `df_clean['year'] = df_clean['date'].dt.year` dan `df_clean['month'] = df_clean['date'].dt.month` digunakan untuk menambahkan kolom baru berupa year dan month berdasarkan data pada kolom date. Penambahan kolom ini bertujuan untuk memperkaya informasi pada dataset sehingga memudahkan analisis data berdasarkan tahun dan bulan transaksi.

Selanjutnya, perintah `print(df_clean[['date', 'year', 'month']].head())` digunakan untuk menampilkan beberapa data awal setelah kolom baru ditambahkan. Output menunjukkan bahwa kolom year dan month berhasil dibuat sesuai dengan nilai pada kolom date.

3.2.5. Data Filtering

Data *filtering* merupakan proses penyaringan data untuk memilih data yang relevan sesuai dengan kebutuhan analisis. Pada tahap ini dilakukan pemilihan data berdasarkan kriteria

tertentu agar data yang digunakan menjadi lebih fokus dan sesuai dengan tujuan analisis. Proses *filtering* bertujuan untuk mengurangi data yang tidak diperlukan serta memastikan bahwa hanya data yang memiliki nilai yang valid dan relevan yang digunakan.

```
# Menampilkan jumlah data sebelum filtering
print("Jumlah data sebelum filtering:", len(df_clean))

# Filter data:
# hanya mengambil transaksi dengan status Delivered to Buyer
df_filter = df_clean[df_clean['status'] == 'Shipped - Delivered to Buyer']

# Menampilkan jumlah data setelah filtering
print("Jumlah data setelah filtering:", len(df_filter))

# Menampilkan 5 data teratas hasil filtering
df_filter.head()
```

```
... Jumlah data sebelum filtering: 19379
Jumlah data setelah filtering: 16672
```

	index	order_id	date	status	fulfilment	sales_channel	ship_service_level	style	sku	category	...	ship_city	ship_state	ship_postal_code
49051	49051	408-4858463-2356347	2022-05-31	Shipped - Delivered to Buyer	Merchant	Amazon.in	Standard	J0385	J0385-KR-M	kurta	...	RAJAHMUNDRY	ANDHRA PRADESH	533120.0
49077	49077	402-7697834-2682766	2022-05-31	Shipped - Delivered to Buyer	Merchant	Amazon.in	Standard	JNE3654	JNE3654-TP-XXL	Top	...	CHENNAI	TAMIL NADU	600042.0
49081	49081	402-7031777-0396341	2022-05-31	Shipped - Delivered to Buyer	Merchant	Amazon.in	Standard	JNE3798	JNE3798-KR-XS	Western Dress	...	NEDUMANGAD	KERALA	695541.0
49082	49082	408-1235521-4337163	2022-05-31	Shipped - Delivered to Buyer	Merchant	Amazon.in	Standard	J0152	J0152-DR-L	Western Dress	...	LEH	JAMMU & KASHMIR	194101.0
49083	49083	403-3845187-4421165	2022-05-31	Shipped - Delivered to Buyer	Merchant	Amazon.in	Standard	J0157	J0157-DR-XL	Western Dress	...	PIMPRI CHINCHWAD	MAHARASHTRA	411027.0

5 rows x 26 columns

Interpretasi:

Kode `df_filter = df_clean[df_clean['status'] == 'Shipped - Delivered to Buyer']` digunakan untuk melakukan penyaringan data dengan mengambil hanya transaksi yang memiliki status Shipped - Delivered to Buyer. Proses ini bertujuan untuk memfokuskan analisis pada transaksi yang telah berhasil dikirim dan diterima oleh pembeli.

Sebelum proses filtering dilakukan, perintah `len(df_clean)` digunakan untuk menampilkan

jumlah seluruh data pada dataset yaitu sebanyak 19.379 data. Setelah filtering dilakukan, perintah `len(df_filter)` menunjukkan bahwa jumlah data berkurang menjadi 16.672 data sesuai dengan kondisi yang telah ditentukan.

Selanjutnya, perintah `df_filter.head()` digunakan untuk menampilkan 5 data teratas hasil filtering. Output menunjukkan bahwa seluruh data yang ditampilkan memiliki status `Shipped - Delivered to Buyer`, sehingga proses filtering berhasil dilakukan sesuai kriteria yang diinginkan.

3.2.6. Data Aggregation

Data *aggregation* merupakan proses pengelompokan dan peringkasan data untuk menghasilkan informasi yang lebih sederhana dan mudah dipahami. Pada tahap ini, data yang memiliki karakteristik yang sama akan dikelompokkan, kemudian dihitung nilai tertentu seperti jumlah atau rata-rata. Proses ini bertujuan untuk melihat pola atau gambaran umum dari data, seperti distribusi jumlah data atau perbandingan nilai antar kelompok. Dengan melakukan agregasi, data yang awalnya detail dapat diringkas menjadi informasi yang lebih jelas dan terstruktur.

```
# Mengelompokkan data berdasarkan kategori
agg_category = df_filter.groupby('category')['amount'].sum().reset_index()

# Mengurutkan dari penjualan terbesar
agg_category = agg_category.sort_values(by='amount', ascending=False)

# Menampilkan hasil agregasi
agg_category
```

```
***
      category  amount
4          Set  5017523.0
6  Western Dress  2894062.0
7         kurta  2490087.0
5          Top   620102.0
2   Ethnic Dress   95554.0
0         Blouse   38063.0
1         Bottom    36811.0
3          Saree    7905.0
```

Interpretasi:

Kode `df_filter.groupby('category')['amount'].sum().reset_index()` digunakan untuk mengelompokkan data berdasarkan kategori produk (`category`) dan menghitung total nilai penjualan (`amount`) pada setiap kategori. Proses ini bertujuan untuk memperoleh ringkasan total penjualan berdasarkan jenis produk.

Selanjutnya, perintah `sort_values(by='amount', ascending=False)` digunakan untuk mengurutkan hasil agregasi dari total penjualan terbesar ke terkecil sehingga kategori dengan penjualan tertinggi dapat lebih mudah diketahui.

Hasil output menunjukkan total penjualan pada masing-masing kategori, seperti kategori Set yang memiliki total penjualan terbesar, diikuti oleh Western Dress, kurta, dan kategori lainnya. Proses agregasi ini membantu dalam menganalisis performa penjualan berdasarkan kategori produk.

3.3. Data Warehouse (Star Schema)

Data warehouse merupakan sistem penyimpanan data yang dirancang untuk mendukung proses analisis dan pengambilan keputusan. Pada penelitian ini, data warehouse dibangun menggunakan model *star schema*, yaitu struktur yang terdiri dari satu tabel fakta (*fact table*) yang terhubung dengan beberapa tabel dimensi (*dimension table*). Model *star schema* dipilih karena memiliki struktur yang sederhana dan mudah digunakan untuk proses analisis. Tabel fakta berisi data utama seperti harga, diskon, *rating*, dan jumlah *review*, sedangkan tabel dimensi berisi informasi pendukung seperti kategori dan produk. Perancangan data warehouse

dilakukan setelah proses transformasi data selesai, sehingga data yang dimasukkan ke dalam tabel sudah dalam kondisi bersih dan terstruktur.

FACT TABLE

```
fact_sales = df[['order_id', 'date', 'sku', 'ship_city', 'status', 'qty', 'amount']]
```

Interpretasi:

Kode `fact_sales = df[['order_id', 'date', 'sku', 'ship_city', 'status', 'qty', 'amount']]` digunakan untuk membuat tabel fakta (fact table) bernama `fact_sales` dari dataset utama. Tabel ini berisi atribut penting yang berkaitan dengan transaksi penjualan, seperti ID pesanan, tanggal transaksi, kode produk, kota pengiriman, status transaksi, jumlah produk, dan total penjualan. Fact table ini digunakan sebagai pusat data transaksi pada data warehouse yang nantinya dapat dianalisis berdasarkan produk, waktu, maupun lokasi.

DIM PRODUCT

```
dim_product = df[['sku', 'category', 'style']].drop_duplicates()
```

Interpretasi:

Kode `dim_product = df[['sku', 'category', 'style']].drop_duplicates()` digunakan untuk membuat tabel dimensi produk (`dim_product`) pada data warehouse. Tabel ini berisi informasi mengenai produk, seperti kode produk (`sku`), kategori produk (`category`), dan jenis atau model produk (`style`).

Perintah `drop_duplicates()` digunakan untuk menghapus data yang duplikat sehingga setiap produk hanya tersimpan satu kali pada tabel dimensi. Tabel dimensi produk ini digunakan untuk mendukung analisis penjualan berdasarkan kategori dan jenis produk.

DIM TIME

```
df['date'] = pd.to_datetime(df['date'])
df['year'] = df['date'].dt.year
df['month'] = df['date'].dt.month

dim_time = df[['date', 'year', 'month']].drop_duplicates()
```

Interpretasi:

Kode `df['date'] = pd.to_datetime(df['date'])` digunakan untuk mengubah kolom `date` menjadi format `datetime` agar data tanggal dapat diproses dengan lebih mudah. Selanjutnya, kode `df['year'] = df['date'].dt.year` dan `df['month'] = df['date'].dt.month` digunakan untuk menambahkan informasi tahun dan bulan berdasarkan kolom tanggal.

Setelah itu, kode `dim_time = df[['date', 'year', 'month']].drop_duplicates()` digunakan untuk membuat tabel dimensi waktu (`dim_time`) yang berisi atribut tanggal, tahun, dan bulan. Perintah `drop_duplicates()` digunakan agar data waktu yang tersimpan tidak memiliki duplikasi. Tabel dimensi waktu ini digunakan untuk mendukung analisis data berdasarkan periode waktu tertentu.

DIM LOCATION

```
dim_location = df[['ship_city', 'ship_state', 'ship_country']].drop_duplicates()
```

Interpretasi:

Kode `dim_location = df[['ship_city', 'ship_state', 'ship_country']].drop_duplicates()` digunakan untuk membuat tabel dimensi lokasi (`dim_location`) pada data warehouse. Tabel ini berisi informasi lokasi pengiriman, seperti kota (`ship_city`), negara bagian (`ship_state`), dan negara (`ship_country`).

Perintah `drop_duplicates()` digunakan untuk menghapus data lokasi yang duplikat sehingga setiap kombinasi lokasi hanya tersimpan satu kali. Tabel dimensi lokasi ini digunakan untuk mendukung analisis penjualan berdasarkan wilayah atau lokasi pengiriman.

3.4. Load

Tahap *load* merupakan proses memasukkan data yang telah melalui tahap transformasi ke dalam struktur data *warehouse* yang telah dirancang sebelumnya. Pada tahap ini, data dimasukkan ke dalam tabel fakta dan tabel dimensi sesuai dengan model *star schema*. Proses *load* dilakukan dengan memindahkan data hasil pengolahan ke dalam tabel dimensi seperti data kategori dan produk, serta ke dalam tabel fakta yang berisi data utama seperti harga, diskon, *rating*, dan jumlah *review*. Tujuan dari tahap ini adalah agar data yang telah bersih dan terstruktur dapat tersimpan dengan baik dalam data *warehouse* sehingga siap digunakan untuk proses analisis.

Load

```
fact_sales.to_csv('fact_salesterbarubangnet.csv', index=False)
dim_product.to_csv('dim_productterbarubangnet.csv', index=False)
dim_time.to_csv('dim_timeterbarubangnet.csv', index=False)
dim_location.to_csv('dim_locationterbarubangnet.csv', index=False)
```

Interpretasi:

Kode `to_csv()` digunakan untuk menyimpan hasil tabel data warehouse ke dalam file berformat CSV. Pada proses

ini, tabel fact_sales, dim_product, dim_time, dan dim_location masing-masing diekspor menjadi file terpisah agar dapat digunakan pada proses analisis atau implementasi data warehouse selanjutnya. Parameter index=False digunakan agar index dataframe tidak ikut tersimpan ke dalam file CSV sehingga struktur data menjadi lebih rapi dan sesuai kebutuhan.

BAB IV

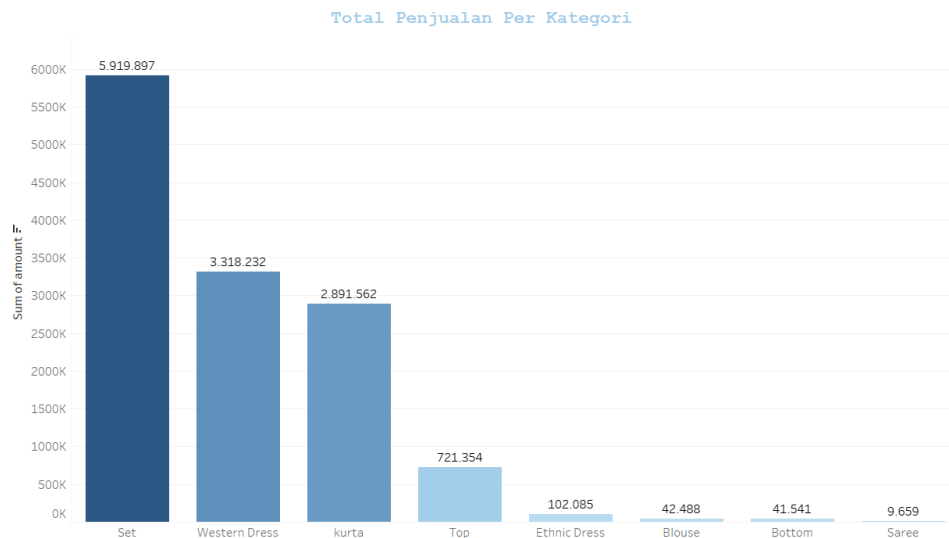
ANALISIS DATA

4.1. Analisis Penjualan Produk per Kategori

Visualisasi ini menggunakan *Bar Chart* untuk menampilkan jumlah produk pada setiap kategori.

- *Axis (X): Category*
- *Values (Y): Total Penjualan (Sum of Amount)*

Visualisasi ini bertujuan untuk mengetahui kategori produk dengan total penjualan tertinggi dan terendah berdasarkan nilai transaksi penjualan.



Gambar 4.1. Analisis Jumlah Produk per Kategori

Berdasarkan grafik, kategori Set memiliki total penjualan paling tinggi dibandingkan kategori lainnya dengan nilai sekitar 5,9 juta. Hal ini menunjukkan bahwa kategori tersebut menjadi kategori produk dengan kontribusi penjualan terbesar serta memiliki tingkat permintaan yang tinggi. Selain itu, kategori Western Dress dan kurta juga memiliki total penjualan yang cukup besar, sehingga dapat dikatakan bahwa kedua kategori tersebut termasuk produk yang banyak diminati oleh pelanggan.

Di sisi lain, kategori seperti Ethnic Dress, Blouse, Bottom, dan Saree memiliki total

penjualan yang relatif lebih rendah dibandingkan kategori lainnya. Hal ini menunjukkan bahwa kontribusi penjualan pada kategori tersebut tidak sebesar kategori utama dan kemungkinan memiliki jumlah transaksi atau minat pelanggan yang lebih sedikit.

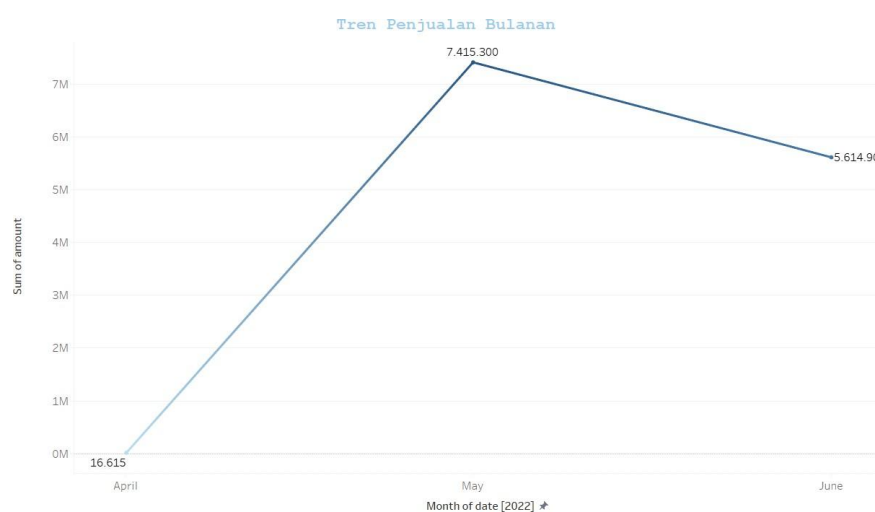
Jika dilihat secara keseluruhan, distribusi total penjualan lebih didominasi oleh beberapa kategori utama, khususnya kategori pakaian seperti Set, Western Dress, dan kurta. Kondisi ini menunjukkan bahwa fokus penjualan pada dataset lebih banyak berada pada kategori-kategori tersebut. Informasi ini dapat dimanfaatkan untuk membantu pengambilan keputusan bisnis, seperti menentukan strategi promosi, pengelolaan stok produk, serta evaluasi kategori produk dengan performa penjualan rendah agar dapat ditingkatkan di masa mendatang.

4.2. Analisis Tren Penjualan Bulanan

Visualisasi ini menggunakan Line Chart untuk menampilkan tren total penjualan berdasarkan bulan pada tahun 2022.

- *Axis (X): Bulan (Month of Date)*
- *Values (Y): Total Penjualan (Sum of Amount)*

Visualisasi ini bertujuan untuk mengetahui perubahan dan perkembangan total penjualan dari waktu ke waktu pada setiap bulan.



Gambar 4.2. Analisis Tren Penjualan Bulanan

Berdasarkan grafik, total penjualan pada bulan April masih sangat rendah dengan nilai sekitar 16 ribu. Selanjutnya, terjadi peningkatan yang sangat signifikan pada bulan Mei dengan

total penjualan mencapai sekitar 7,4 juta. Peningkatan ini menunjukkan adanya lonjakan transaksi penjualan yang sangat besar dibandingkan bulan sebelumnya.

Pada bulan Juni, total penjualan mengalami penurunan menjadi sekitar 5,6 juta. Meskipun mengalami penurunan dibandingkan bulan Mei, nilai penjualan pada bulan Juni masih tergolong tinggi dibandingkan bulan April. Hal ini menunjukkan bahwa aktivitas penjualan tetap berada pada tingkat yang cukup baik.

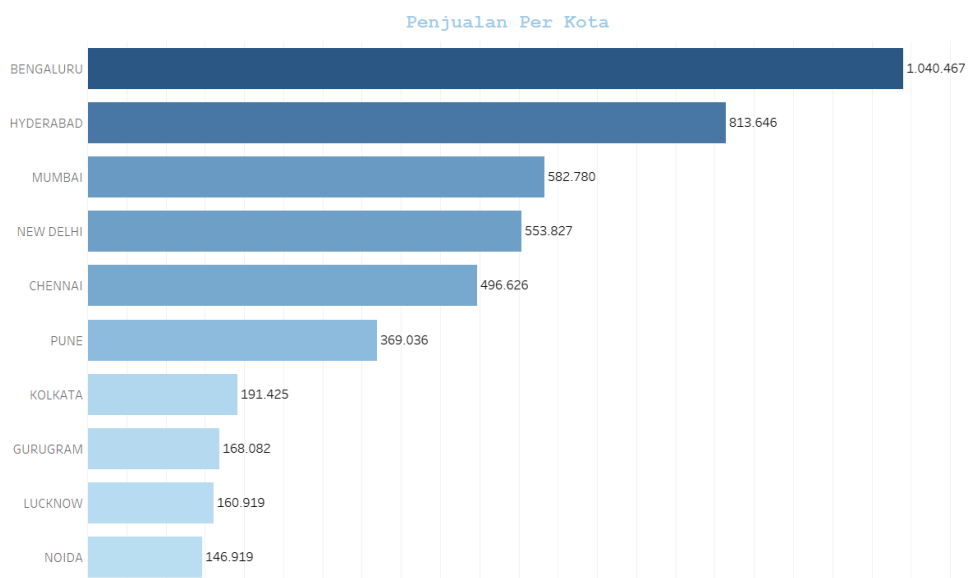
Jika dilihat secara keseluruhan, tren penjualan menunjukkan pola peningkatan tajam dari April ke Mei, kemudian mengalami penurunan pada Juni. Informasi ini dapat digunakan untuk menganalisis pola penjualan bulanan, mengevaluasi faktor yang memengaruhi kenaikan maupun penurunan penjualan, serta membantu dalam pengambilan keputusan bisnis seperti strategi promosi dan perencanaan stok produk pada periode tertentu.

4.3. Analisis Top Penjualan Per-Kota

Visualisasi ini menggunakan Horizontal Bar Chart untuk menampilkan total penjualan pada beberapa kota.

- *Axis (X): Kota (City)*
- *Values (Y): Total Penjualan (Sum of Amount)*

Visualisasi ini bertujuan untuk mengetahui kota dengan kontribusi penjualan tertinggi dan membandingkan performa penjualan antar kota.



Gambar 4.3. Analisis Top Penjualan Per-Kota

Berdasarkan grafik, kota BENGALURU memiliki total penjualan tertinggi dengan nilai sekitar 1,04 juta. Hal ini menunjukkan bahwa kota tersebut menjadi wilayah dengan kontribusi penjualan terbesar dibandingkan kota lainnya. Selanjutnya, kota HYDERABAD berada pada posisi kedua dengan total penjualan sekitar 813 ribu, diikuti oleh MUMBAI dan NEW DELHI yang juga memiliki nilai penjualan cukup tinggi.

Di sisi lain, kota seperti GURUGRAM, LUCKNOW, dan NOIDA memiliki total penjualan yang relatif lebih rendah dibandingkan kota lainnya. Hal ini menunjukkan bahwa kontribusi penjualan pada kota-kota tersebut tidak sebesar wilayah utama seperti Bengaluru dan Hyderabad.

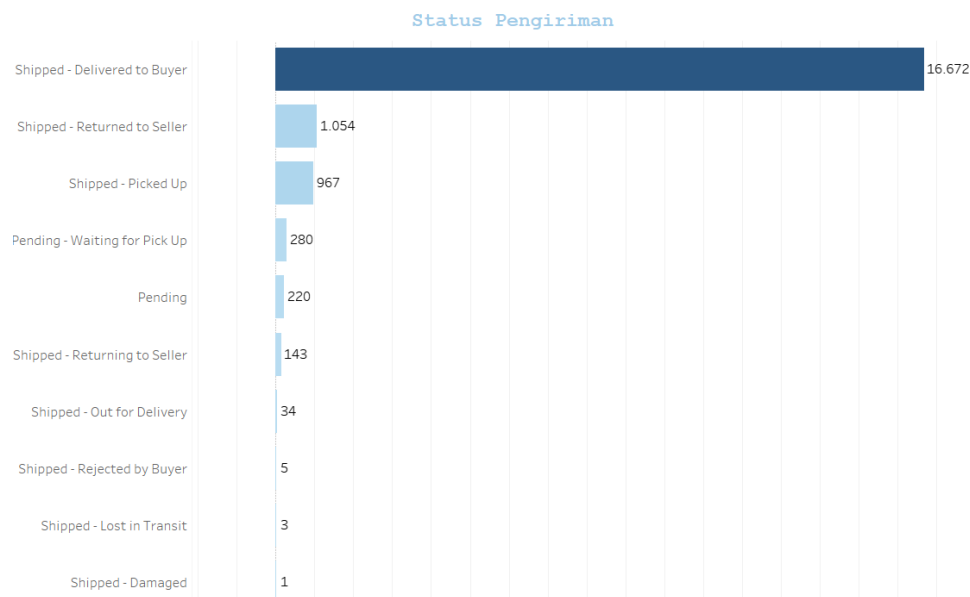
Jika dilihat secara keseluruhan, distribusi penjualan menunjukkan bahwa beberapa kota besar memiliki dominasi penjualan yang lebih tinggi dibandingkan kota lainnya. Informasi ini dapat dimanfaatkan untuk membantu perusahaan dalam menentukan strategi pemasaran, distribusi produk, serta pengelolaan stok berdasarkan wilayah dengan tingkat penjualan tertinggi maupun terendah.

4.4. Analisis Status Pengiriman

Visualisasi ini menggunakan Horizontal Bar Chart untuk menampilkan jumlah transaksi berdasarkan status pengiriman.

- *Axis (X): Status Pengiriman (Status)*
- *Values (Y): Jumlah Transaksi (Count of Status)*

Visualisasi ini bertujuan untuk mengetahui distribusi status pengiriman serta melihat status transaksi yang paling banyak terjadi pada dataset.



Gambar 4.4. Analisis Status Pengiriman

Berdasarkan grafik, status Shipped – Delivered to Buyer memiliki jumlah transaksi paling tinggi dengan total sekitar 16.672 transaksi. Hal ini menunjukkan bahwa sebagian besar pesanan berhasil dikirim dan diterima oleh pembeli, sehingga proses pengiriman pada dataset secara umum berjalan dengan baik.

Selain itu, terdapat beberapa status lain seperti Shipped - Returned to Seller dan Shipped - Picked Up yang juga memiliki jumlah transaksi cukup banyak dibandingkan status lainnya. Hal ini menunjukkan bahwa terdapat sebagian pesanan yang mengalami pengembalian atau masih berada dalam proses pengiriman.

Di sisi lain, status seperti Shipped - Rejected by Buyer, Shipped - Lost in Transit, dan Shipped - Damaged memiliki jumlah transaksi yang sangat sedikit. Kondisi ini menunjukkan bahwa kasus kehilangan barang, kerusakan barang, maupun penolakan oleh pembeli relatif jarang terjadi.

Jika dilihat secara keseluruhan, distribusi status pengiriman didominasi oleh transaksi yang berhasil diterima oleh pembeli. Informasi ini dapat digunakan untuk mengevaluasi performa proses pengiriman, tingkat keberhasilan transaksi, serta mengidentifikasi potensi masalah pada proses distribusi dan pengiriman barang.

BAB V

PENUTUP

5.1. Kesimpulan

Berdasarkan hasil perancangan dan analisis yang telah dilakukan, dapat disimpulkan bahwa implementasi sistem Business Intelligence pada dataset penjualan e-commerce ini telah memberikan gambaran yang komprehensif mengenai performa bisnis perusahaan. Proses ETL (Extract, Transform, Load) yang dijalankan terbukti mampu mengubah data mentah yang tidak terstruktur menjadi informasi yang bersih, konsisten, dan siap diolah. Melalui penerapan model star schema, integrasi antara tabel fakta dan tabel dimensi memudahkan proses identifikasi pola penjualan, di mana kategori produk "Set" muncul sebagai penyumbang pendapatan terbesar. Selain itu, terlihat adanya lonjakan transaksi yang signifikan pada bulan Mei 2022 dengan pusat aktivitas ekonomi tertinggi berada di kota Bengaluru. Secara operasional, sistem pengiriman juga menunjukkan performa yang sangat baik dengan mayoritas pesanan berhasil diterima oleh pelanggan tanpa kendala teknis yang berarti.

5.2. Saran

Bertolak dari hasil analisis tersebut, perusahaan disarankan untuk memperkuat strategi pemasaran dan manajemen stok pada produk-produk unggulan seperti kategori "Set" dan "Western Dress" guna menjaga momentum pertumbuhan. Perlu juga dilakukan penelusuran lebih lanjut terkait penurunan tren penjualan di bulan Juni agar langkah antisipasi dapat segera dipersiapkan. Untuk pengembangan ke depannya, sistem ini dapat ditingkatkan dengan menambahkan indikator kinerja utama (KPI) yang lebih spesifik serta membangun dashboard visualisasi yang interaktif guna mendukung pengambilan keputusan yang lebih taktis dan cepat. Selain itu, perluasan analisis pada wilayah dengan tingkat penjualan rendah dapat menjadi peluang untuk menemukan segmen pasar baru yang belum tergarap secara maksimal.